SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.	3
2	An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.	3
3	A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.	3

4	A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.	3
5	A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.	3
6	A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.	3
7	A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.	3
8	An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.	3
9	A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.	3

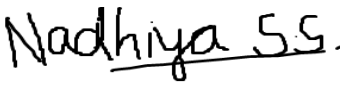
10	A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.	3
11	A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.	3
12	A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.	3
13	A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.	3
14	A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.	3
15	A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.	3
16	A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.	3

17	A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.	3
18	A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.	3
19	A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.	3
20	A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.	3

Code of Conduct:

I Nadhiya SS(192321136) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

Q1. Testing as an Engineering Activity

Answer:

Software testing should be performed as a planned engineering activity rather than through random or ad-hoc testing. A systematic approach helps the team achieve better coverage and identify defects at an early stage.

1. Systematic Test Design

The team should first understand the requirements and then prepare test cases using suitable testing techniques.

Important techniques include:

- **Equivalence Partitioning:** Divides input data into valid and invalid groups.
- **Boundary Value Analysis:** Tests values at the boundaries of input ranges.
- **Decision Table Testing:** Tests different combinations of conditions.
- **State Transition Testing:** Verifies changes between different system states.
- **Use Case Testing:** Tests software according to real user activities.

2. Testing Tools

Testing tools can be used to improve efficiency and reduce manual effort. Tools may support:

- Test case management
- Automated testing
- Defect tracking
- Regression testing
- Test reporting

For example, automated tests can repeatedly check login, balance enquiry, fund transfer, and transaction validation.

3. Standards and Procedures

The organization should define standard procedures for:

- Test planning
- Test-case preparation
- Test execution
- Defect reporting
- Retesting
- Regression testing

Each defect should contain details such as severity, priority, steps to reproduce, expected result, and actual result.

4. Practical Example

For a banking fund-transfer feature, the tester can prepare test cases such as:

Test Condition	Expected Result
Valid account and sufficient balance	Transfer succeeds
Invalid account number	Error message is displayed
Insufficient balance	Transfer is rejected
Zero or negative amount	Validation message appears
Amount exceeds daily limit	Transfer is rejected
Network interruption	Transaction is handled safely

5. Benefits of Systematic Testing

Systematic testing ensures that important requirements and boundary conditions are not ignored. It also makes testing repeatable and helps detect defects introduced after changes.

Conclusion:

Replacing random testing with systematic techniques, suitable tools, and defined standards improves test coverage, defect detection, reliability, and overall software quality.

Q2. Structured Testing Process for an ATM System

Answer:

An ATM performs critical operations such as authentication, cash withdrawal, balance checking, and transaction processing. Therefore, testing should be conducted through a clearly defined process throughout the SDLC.

1. Requirement Analysis

The tester identifies the expected ATM behaviour.

For withdrawal functionality:

- A valid PIN must be entered.
- The account must contain sufficient funds.
- The withdrawal amount must be within the permitted limit.
- The correct cash amount must be dispensed.
- The account balance must be updated.

2. Test Planning

The test plan should specify:

- Testing objectives
- Testing scope
- Resources
- Test cases
- Schedule
- Expected results

Both successful and unsuccessful withdrawal scenarios should be included.

3. Test Case Design

Example test cases:

Test Case	Condition	Expected Result
TC01	Valid PIN and sufficient balance	Cash is dispensed
TC02	Incorrect PIN	Transaction is rejected
TC03	Insufficient balance	Withdrawal is rejected
TC04	Amount exceeds limit	Transaction is rejected
TC05	Invalid amount	Error message is displayed

4. Test Execution

The prepared test cases are executed and the actual results are compared with the expected results.

For example, if a customer requests ₹5,000 and has sufficient funds, the ATM should dispense ₹5,000 and deduct exactly ₹5,000 from the account.

5. Defect Reporting

If the actual result differs from the expected result, a defect should be recorded with:

- Test case ID
- Steps to reproduce
- Expected result
- Actual result
- Severity
- Priority

6. Retesting and Regression Testing

After a defect is corrected, **retesting** confirms that the particular problem has been fixed.

Regression testing checks whether the correction has affected other functions such as balance enquiry or transaction history.

7. Test Closure

At the end of testing, the team reviews test results, defects, coverage, and overall quality before deciding whether the system is ready.

Conclusion:

Following a structured SDLC-based testing process provides better coverage, fewer missed defects, reliable ATM transactions, and improved customer confidence.

Q3. Errors, Defects, and Failures in a Web Application

Answer:

When a web application produces an incorrect result, it is important to distinguish between an **error, defect, and failure**. These terms describe different stages of a software problem.

1. Error

An error is a mistake made by a person during requirements analysis, design, coding, or other development activities.

Example:

A developer misunderstands the login requirement and implements incorrect password-validation logic.

2. Defect

A defect is a flaw introduced into the software because of an error.

Example:

Expected:

Login should succeed only when both username and password are correct.

Actual implementation:

The application checks only the username.

The incorrect authentication logic is a defect.

3. Failure

A failure occurs when the defective software executes and produces an incorrect result.

Example:

A user enters an incorrect password but is still allowed to log in.

Relationship

The relationship can be represented as:

Human Error → Software Defect → Failure during Execution

For example, an incorrect understanding of a transaction requirement may result in defective calculation logic. When a customer performs the transaction, the system may produce an incorrect balance.

Login Example

Situation	Expected Result	Actual Result
Correct username + correct password	Login successful	Login successful
Correct username + wrong password	Login rejected	Login successful

The second situation represents a failure.

Transaction Example

Suppose an account contains ₹10,000 and the user transfers ₹3,000.

Expected balance:

₹10,000 − ₹3,000 = ₹7,000

If the application deducts ₹4,000, the observed behaviour is a failure caused by a possible defect in transaction logic.

Improving Defect Reporting

A good defect report should include:

- Defect ID
- Affected module
- Steps to reproduce
- Expected result
- Actual result
- Severity
- Priority
- Supporting screenshots or logs

Conclusion:

Errors, defects, and failures are connected but have different meanings. Understanding these terms helps testers communicate problems clearly and enables developers to resolve them effectively.

Q4. Defect Clustering and Test Case Prioritization

Answer:

Defect clustering is a testing principle which states that a small number of modules often contain a large number of defects. Therefore, testing effort should be concentrated on high-risk and defect-prone areas.

1. Identify Defect-Prone Modules

The testing team should analyze previous defect reports and identify modules having:

- High defect counts
- Frequent changes
- Complex logic
- Security risks
- Large numbers of dependencies

2. Apply Defect Clustering

If most defects are found in a few modules, these modules should receive additional testing attention. For example, in an e-commerce system, payment and checkout modules may contain more defects than simple information pages.

3. Prioritize Test Cases

Test cases can be prioritized according to:

- Business importance
- Risk level
- Defect history
- Complexity
- Frequency of use
- Impact of failure

High-priority test cases should be executed first.

4. Example

Suppose a project contains five modules:

Module	Previous Defects	Priority
Login	8	High
Payment	20	Very High
Product Search	5	Medium
Profile	2	Low
Help	1	Low

The payment module should receive the greatest testing attention because it has the highest defect concentration and business impact.

5. Benefits

Applying defect clustering helps:

- Focus testing effort

- Detect critical defects earlier
- Reduce testing time
- Improve risk coverage
- Avoid unnecessary testing of low-risk areas

Conclusion:

Defect clustering helps testers concentrate resources on areas where defects are more likely to occur. Combining this principle with risk-based test prioritization improves testing efficiency and software quality.

Q5. Defect Classification and Defect Repository

Answer:

A defect repository stores information about software defects discovered during testing. If an organization does not use this information effectively, the same problems may appear repeatedly in later releases.

1. Origin of Defects

Defects can originate from:

- Incorrect requirements
- Design mistakes
- Coding errors
- Incorrect data
- Interface problems
- Integration issues
- Configuration problems

2. Classification of Defects

Defects can be classified according to their nature, such as:

- Functional defects
- Performance defects
- Interface defects
- Data defects
- Security defects
- Calculation defects
- Integration defects

Classification makes defect analysis easier.

3. Using the Defect Repository

The repository should record information such as:

- Defect description
- Module
- Severity
- Priority
- Root cause
- Status
- Date reported
- Resolution
- Related test cases

4. Supporting Test Design

Historical defect information can be used to create better test cases.

For example, if previous releases repeatedly had calculation defects in the billing module, testers should add more boundary, calculation, and negative test cases for that module.

5. Preventing Recurring Defects

The team should:

- Perform root-cause analysis.
- Update existing test cases.
- Add regression tests.
- Conduct code reviews.
- Improve requirements.
- Monitor repeated defect patterns.

Conclusion:

A defect repository should not only store problems but also provide knowledge for future testing. Proper classification and historical analysis can reduce repeated defects and improve software quality.

Q6. Identifying the Root Cause of Failures in a Mobile Banking Application

Answer:

A mobile banking application must handle financial transactions safely. Frequent crashes during fund transfers may be caused by coding errors, network interruptions, incorrect input handling, or integration problems.

1. Identify Possible Causes

Possible causes include:

- Incorrect fund-transfer logic
- Network failures
- Invalid input handling
- Server communication problems
- Database errors
- Memory or performance issues

2. Apply Systematic Testing

Different testing levels should be applied:

UnitTesting:

Tests individual functions such as balance checking.

IntegrationTesting:

Checks communication between the mobile application, server, and database.

FunctionalTesting:

Verifies that fund transfer works according to requirements.

PerformanceTesting:

Checks application behaviour under heavy usage.

RegressionTesting:

Ensures that fixes do not introduce new defects.

3. Suitable Test Cases

Test Case	Condition	Expected Result
TC01	Valid account and sufficient balance	Transfer succeeds
TC02	Insufficient balance	Transfer is rejected
TC03	Invalid beneficiary	Error message appears
TC04	Internet interruption	Application handles interruption safely
TC05	Transfer button clicked repeatedly	Duplicate transaction is prevented
TC06	Slow server response	Application does not crash

4. Practical Example

Suppose a customer attempts to transfer ₹5,000 and the application crashes because the server response is delayed.

The tester can reproduce the problem under different network conditions and examine logs. If the cause is improper timeout handling, the developer can correct the error-handling logic.

After the correction, the tester performs retesting and regression testing.

5. Preventive Improvements

The team should:

- Test different network conditions.
- Perform automated regression testing.
- Validate user inputs.
- Monitor crash logs.
- Test duplicate transactions.
- Perform integration testing.
- Conduct code reviews.

Conclusion:

Systematic testing and root-cause analysis help identify the reasons behind mobile banking failures. Combining functional, integration, performance, and regression testing improves application reliability.

Q7. Difference Between Testing and Debugging

Answer:

Testing and debugging are related software quality activities, but their objectives are different. Testing identifies problems, whereas debugging investigates and fixes their causes.

1. Software Testing

Testing involves executing or evaluating software to determine whether it satisfies its requirements.

Example:

A tester enters an incorrect password and discovers that the system still allows login. The tester reports the defect.

2. Software Debugging

Debugging involves locating, analyzing, and correcting the cause of a reported problem.

Example:

A developer investigates the login problem and discovers that the password-validation condition is incorrect. The developer fixes the code.

3. Difference Between Testing and Debugging

Basis	**Testing**	**Debugging**
Purpose	Finds failures and defects	Finds and fixes causes
Main activity	Executes test cases	Examines and modifies code

Usually performed by	Testers/developers	Mainly developers
Starting point	Requirements and test cases	Reported failure
Output	Test results and defect report	Corrected software
Main focus	Detecting problems	Resolving problems

4. Effects of Confusing Both

If developers and testers confuse their responsibilities:

- Defects may not be reported properly.
- Root causes may not be investigated.
- Testing time may increase.
- Fixes may not be verified.
- New defects may remain undetected.

5. Structured Approach

A good process is:

Test → Identify Failure → Report Defect → Debug → Fix → Retest → Regression Test

For example, if an ATM deducts ₹5,000 but dispenses only ₹4,000, the tester reports the problem. The developer investigates the cause, fixes the code, and the tester verifies the correction.

Conclusion:

Testing and debugging should work together but should not be treated as the same activity. Proper coordination between both improves defect resolution and software quality.

Q8. Cross-Browser Testing for an E-Commerce Platform

Answer:

Cross-browser testing verifies that an application works correctly on different browsers, operating systems, and screen sizes. E-commerce websites may behave differently because browsers use different rendering engines and JavaScript or CSS implementations.

1. Causes of Cross-Browser Defects

Common causes include:

- CSS compatibility problems
- JavaScript differences
- Unsupported HTML features
- Browser-version differences
- Responsive-design problems

2. Cross-Browser Testing Strategy

The team should test commonly supported browsers such as:

- Google Chrome
- Mozilla Firefox
- Microsoft Edge
- Safari

Testing should cover:

- Login
- Product search
- Product details
- Shopping cart
- Checkout
- Payment
- User interface

3. Example Test Cases

Test Case	Condition	Expected Result
TC01	Open site in Chrome	Website loads correctly
TC02	Open site in Firefox	Layout and functions work
TC03	Add product using Edge	Product is added
TC04	Checkout using Safari	Checkout succeeds
TC05	Open site on mobile	Layout adapts correctly

4. Tools

Useful tools include:

- **Selenium:** Browser automation
- **Playwright:** Multi-browser automated testing
- **Browser Developer Tools:** Helps identify UI and JavaScript problems
- **Cloud testing platforms:** Test browsers and devices remotely

5. Improving Reliability

Systematic cross-browser testing identifies compatibility problems before release.

For example, if a checkout button works correctly in Chrome but does not respond in Firefox, testing can identify the issue before customers experience it.

Conclusion:

Cross-browser testing ensures that customers receive a consistent experience regardless of their supported browser or device. Combining manual testing, automation, and compatibility testing provides better coverage.

Q9. Defect Classification in a Payroll System

Answer:

A payroll application calculates salaries using information such as basic salary, allowances, deductions, attendance, and taxes. A logical error can result in incorrect salary payments.

1. Types of Defects

Payroll defects may include:

- Logical defects
- Calculation defects
- Functional defects
- Data defects
- Integration defects
- Interface defects

2. Severity of Defects

Defects should also be classified according to their impact.

For example, an incorrect salary calculation is generally a high-impact problem because it can affect employee payments.

3. Practical Example

Suppose:

- Basic salary = ₹30,000
- Allowance = ₹5,000

Expected salary:

₹30,000 + ₹5,000 = ₹35,000

If the application calculates ₹33,000, there is a calculation or logical defect.

4. Test Cases

Test Case	Condition	Expected Result
TC01	Basic salary + allowance	Correct total salary
TC02	Salary with deductions	Correct deduction
TC03	Different attendance values	Correct salary adjustment

TC04	Tax calculation	Correct tax amount
TC05	Boundary salary values	Correct result

5. Prevention

To prevent similar defects, the organization should:

- Perform unit testing.
- Verify calculations independently.
- Validate employee data.
- Test integration between attendance and payroll.
- Maintain a defect repository.
- Perform regression testing after fixes.

Conclusion:

Defect classification helps the testing team understand the nature and impact of payroll problems. Proper classification, detailed test cases, and regression testing improve salary calculation accuracy and system reliability.

Q10. Testing as an Engineering Activity

Answer:

Testing as an engineering activity means performing software testing through a planned, systematic, measurable, and documented process. Random testing may overlook important scenarios and therefore should be replaced with structured techniques.

1. Understand Requirements

The testing team should first study the requirements and identify expected system behaviour.

For example, an online shopping application may require users to:

- Log in
- Search products
- Add products to the cart
- Make payments
- View order details

These requirements provide the foundation for test design.

2. Prepare a Test Plan

The test plan should specify:

- Testing objectives
- Scope

- Features to be tested
- Resources
- Schedule
- Risks
- Entry and exit criteria

3. Design Test Cases

Test cases should include inputs, execution steps, and expected results.

Suitable techniques include:

- **Equivalence Partitioning**
- **Boundary Value Analysis**
- **Decision Table Testing**
- **State Transition Testing**

These techniques provide better coverage than random testing.

4. Use Tools and Standards

Testing tools can help automate repetitive activities, manage test cases, and track defects.

For example:

- Selenium or Playwright can automate browser testing.
- Defect-management systems can track issues.
- Test-management tools can organize test cases and results.

The team should also follow common procedures for execution, defect reporting, retesting, and regression testing.

5. Practical Example

Consider an online banking application with a fund-transfer feature.

Test Case	Condition	Expected Result
TC01	Valid beneficiary and balance	Transfer succeeds
TC02	Insufficient balance	Transfer rejected
TC03	Zero transfer amount	Validation message appears
TC04	Amount exceeds daily limit	Transfer rejected
TC05	Network interruption	Transaction handled safely
TC06	Repeated transfer request	Duplicate transaction prevented

6. Benefits of Structured Testing

A systematic approach:

- Improves test coverage.

- Tests boundary conditions.
- Covers invalid inputs.
- Makes test execution repeatable.
- Helps reproduce defects.
- Makes test progress measurable.
- Supports regression testing.

7. Overall Improvement

Engineering-based testing reduces dependence on random testing and provides a consistent process for identifying, reporting, fixing, and verifying defects.

Conclusion:

Software testing becomes more effective when it is treated as a systematic engineering activity. Proper requirements analysis, test planning, test design, tools, standards, and regression testing improve defect detection and overall software quality.

Q11. Data Inconsistency in a Healthcare Application

Answer:

A healthcare application contains important information such as patient details, medical records, appointments, prescriptions, and billing information. If the data is inconsistent across modules, it can lead to incorrect decisions and affect the reliability of the application.

1. Identify the Problem

Data inconsistency may occur when the same information is stored or displayed differently in different modules.

For example, a patient's blood group may be recorded as **O+** in the registration module but displayed as **A+** in the medical-record module.

2. How Defects Originate

Defects may originate during development because of:

- Incorrect requirements
- Programming errors
- Incorrect database operations
- Improper data validation
- Synchronization problems
- Integration defects
- Incorrect update logic

- Different data formats

3. Systematic Testing

The following testing activities can be applied:

Functional Testing:

Checks whether each module handles data correctly.

Integration Testing:

Verifies that information is transferred correctly between modules.

Database Testing:

Checks whether stored data matches the expected values.

Regression Testing:

Ensures that fixing one defect does not create another problem.

4. Practical Example

Suppose a patient's address is updated in the registration module but the old address continues to appear in the billing module.

The tester should:

1. Verify the original input.
2. Check the database record.
3. Verify communication between modules.
4. Compare expected and actual data.
5. Report the defect.
6. Retest after correction.

5. Improvements

The organization should:

- Use proper input validation.
- Maintain common data formats.
- Perform database testing.
- Perform integration testing.
- Maintain traceable test cases.
- Conduct regression testing.
- Analyze repeated data-related defects.

Conclusion:

Systematic functional, integration, database, and regression testing helps maintain data integrity. Proper validation and defect tracking can prevent inconsistent healthcare information.

Q12. Using a Defect Repository to Prevent Repeated Defects

Answer:

A defect repository stores information about defects identified during testing and development. Repeated defects indicate that previous defect information is not being properly analyzed and converted into future testing activities.

1. Analyze the Defect Repository

The team should review historical defect information to identify:

- Frequently affected modules
- Repeated defect types
- Common root causes
- High-risk features
- Defects occurring across multiple releases

2. Support Test Design

Previous defects can be converted into new test cases.

For example, if a payment module repeatedly produces incorrect calculations, additional calculation, boundary, and negative test cases should be included in future releases.

3. Identify Defect Patterns

Patterns can be identified by analyzing:

- Defect frequency
- Defect severity
- Defect priority
- Affected modules
- Root causes
- Release-wise defect counts

For example, if most defects occur in the payment module after every release, it should be considered a high-risk area.

4. Root-Cause Analysis

The team should determine whether repeated defects are caused by:

- Poor requirements
- Incorrect coding
- Incomplete test cases
- Integration problems

- Insufficient validation
- Lack of regression testing

5. Prevent Recurring Defects

The company should:

- Update existing test cases.
- Add regression test cases.
- Perform root-cause analysis.
- Conduct code reviews.
- Improve requirements.
- Prioritize high-risk modules.
- Automate repetitive tests.

6. Expected Benefits

Effective use of a defect repository results in:

- Fewer repeated defects
- Better test coverage
- Faster defect detection
- Improved test planning
- Better software quality

Conclusion:

A defect repository should be treated as a source of knowledge rather than simply a storage location. Analyzing historical defect patterns and using them in test design helps prevent recurring problems.

Q13. Testing a Web-Based Learning System During Peak Usage

Answer:

A web-based learning system may work correctly under normal conditions but fail when a large number of students access it simultaneously. This indicates gaps in performance testing, test planning, and test coverage.

1. Identify Testing Gaps

Possible gaps include:

- Lack of load testing
- Insufficient performance testing
- Testing with only a small number of users
- No peak-usage scenarios

- Lack of stress testing
- Insufficient server testing
- Incomplete database testing

2. Importance of Test Planning

The test plan should include different usage conditions:

- Normal usage
- Peak usage
- Multiple simultaneous logins
- Simultaneous assignment submission
- Online examinations
- Large file uploads

3. Performance Testing

Important performance measurements include:

- Response time
- Throughput
- Resource utilization
- Error rate
- Number of concurrent users

Load testing checks the system under the expected number of users.

Stress testing checks the system beyond its expected capacity.

4. Practical Example

Suppose an online examination system works properly with 50 students but becomes very slow when 1,000 students start an examination at the same time.

The testing team should reproduce this condition using controlled load testing.

Possible causes may include:

- Server capacity
- Database performance
- Network resources
- Application logic
- Incorrect configuration

5. Improve Test Coverage

The team should include:

- Normal usage scenarios

- Peak traffic
- Multiple simultaneous logins
- Simultaneous submissions
- Online examinations
- Large file uploads
- Database-intensive operations
- Network interruptions

6. Benefits

Improved performance testing can:

- Detect bottlenecks early.
- Improve response time.
- Reduce system failures.
- Increase reliability.
- Improve user experience.

Conclusion:

Peak-usage failures can be reduced by including realistic performance scenarios in the test plan. Load testing, stress testing, and better coverage help make the learning system more reliable.

Q14. Early Involvement of Testers in the SDLC

Answer:

Testers should be involved from the early stages of the Software Development Life Cycle rather than participating only after development is completed. Early involvement helps identify problems before they become costly defects.

1. Requirement Stage

Testers can review requirements and identify:

- Ambiguous requirements
- Missing conditions
- Conflicting requirements
- Requirements that are difficult to test

For example, instead of saying **"the system should respond quickly,"** the requirement can specify an exact response-time limit.

2. Design Stage

Testers can review the design and identify:

- Integration risks
- Interface problems
- Missing error handling
- Security concerns
- Difficult-to-test functions

3. Development Stage

Testers can work with developers to:

- Understand new features
- Prepare test cases
- Prepare test data
- Identify risks
- Plan integration testing

4. Testing Stage

Once the software is available, testers perform:

- Test execution
- Defect reporting
- Retesting
- Regression testing
- Test-result analysis

5. Practical Example

Consider a banking application with the requirement:

"Users should be able to transfer money securely."

An early-involved tester can ask:

- What happens if the balance is insufficient?
- What is the maximum transfer amount?
- What happens during network failure?
- How are duplicate transactions prevented?

These questions help developers clarify the requirements before implementation.

6. Benefits

Early tester involvement provides:

- Early defect detection
- Lower cost of fixing defects
- Better requirements

- Improved test coverage
- Better communication
- Reduced misunderstandings

7. Team Collaboration

The organization should encourage regular:

- Requirement reviews
- Design reviews
- Test planning meetings
- Defect discussions
- Developer-tester communication

Conclusion:

Testers should be considered part of the development process from the beginning. Early collaboration improves requirements, reduces defects, and produces more reliable software.

Q15. Improving Test Coverage and Validation

Answer:

Incomplete test coverage occurs when only a limited number of inputs, conditions, or execution paths are tested. A system may work correctly for normal cases but fail under unusual conditions.

1. Identify Coverage Gaps

The tester should look for missing:

- Invalid inputs
- Boundary values
- Error conditions
- User roles
- Alternative execution paths
- Integration scenarios
- Exceptional conditions

2. Test Case Design Techniques

Important techniques include:

Equivalence Partitioning:

Divides input values into valid and invalid groups.

Boundary Analysis:

Tests values at and around the limits.

Value

Decision Table Testing:

Tests different combinations of conditions.

State Transition Testing:

Checks how the system behaves when moving between different states.

3. Practical Example

Suppose a banking system accepts transfers between **₹1,000 and ₹50,000**.

Testing only ₹10,000 is insufficient.

Test Case	Input	Expected Result
TC01	₹10,000	Transfer accepted
TC02	₹999	Transfer rejected
TC03	₹1,000	Transfer accepted
TC04	₹50,000	Transfer accepted
TC05	₹50,001	Transfer rejected
TC06	₹0	Transfer rejected
TC07	Negative amount	Transfer rejected

4. Validation

Validation compares:

Expected Result vs Actual Result

If there is a difference, the tester should record and investigate the problem.

5. Improve Test Coverage

The team should:

- Review requirements.
- Map requirements to test cases.
- Add boundary tests.
- Add invalid-input tests.
- Prioritize high-risk features.
- Perform integration testing.
- Perform regression testing.
- Review defects from previous releases.

6. Benefits

Better test coverage helps:

- Find hidden defects.

- Reduce production failures.
- Improve reliability.
- Increase confidence.
- Validate important requirements.

Conclusion:

Proper test coverage ensures that the system is tested under normal, boundary, invalid, and exceptional conditions. Systematic test-case design improves reliability and reduces missed defects.

Q16. Improving Defect Reporting in a Logistics Application

Answer:

Defect reporting is an important part of software testing because it provides clear communication between testers and developers. Poor reporting can delay defect identification and correction.

1. Understand Defect Definitions

A **defect** is a flaw in the software that may cause incorrect behaviour.

A clear defect report should explain what was expected and what actually happened.

2. Important Details in a Defect Report

A structured report should include:

- Defect ID
- Defect title
- Module name
- Environment
- Steps to reproduce
- Expected result
- Actual result
- Severity
- Priority
- Screenshots or logs
- Current status

3. Practical Example

Suppose a logistics application shows the wrong delivery status.

A poor report would be:

“Delivery status is wrong.”

A better report would be:

Expected: Order should show "Delivered" after successful delivery confirmation.

Actual: Order continues to show "Out for Delivery."

Steps:

1. Open the order.
2. Confirm delivery.
3. Refresh the order page.
4. Observe the status.

This gives developers enough information to reproduce the problem.

4. Structured Reporting

A standardized defect-reporting format improves:

- Communication
- Traceability
- Reproduction
- Prioritization
- Resolution time

5. Improvements

The organization should:

- Use a standard defect template.
- Train testers in defect reporting.
- Clearly define severity and priority.
- Attach screenshots and logs.
- Avoid vague descriptions.
- Maintain a defect repository.
- Track defects until closure.

6. Benefits

Good defect reporting results in:

- Faster defect identification
- Faster correction
- Better communication
- Easier tracking
- Improved testing efficiency

Conclusion:

Clear and structured defect reports help developers understand and reproduce problems quickly. Standard reporting practices improve communication and reduce the time required to resolve defects.

Q17. Maintaining Consistency in Testing Across Modules

Answer:

Testing consistency means following common procedures, standards, and methods across different modules. Without a standardized process, different testers may test the same type of feature in different ways.

1. Identify the Problem

Inconsistent testing may occur because of:

- Different testing methods
- Lack of standard procedures
- Different test-case formats
- Inconsistent defect reporting
- Poor communication
- Different interpretation of requirements

2. Standardized Testing Process

The organization should define a common process:

Requirement Analysis → Test Planning → Test Design → Test Execution → Defect Reporting → Retesting → Regression Testing

Every module should follow this basic process.

3. Standard Test Case Format

Each test case should contain:

- Test case ID
- Objective
- Preconditions
- Test steps
- Input
- Expected result
- Actual result
- Status

4. Practical Example

Suppose an e-commerce application contains login, payment, and order modules.

All three modules should use the same approach for:

- Test-case documentation
- Severity classification
- Defect reporting
- Retesting
- Regression testing

This creates consistency across the project.

5. Process Reviews

The team should conduct:

- Test-case reviews
- Requirement reviews
- Defect reviews
- Test-result reviews

These activities help identify differences in testing practices.

6. Methods to Ensure Consistency

The organization can use:

- Standard operating procedures
- Common templates
- Test-management tools
- Automation frameworks
- Tester training
- Checklists
- Regular reviews

Conclusion:

Standardized testing processes improve consistency across modules. Common procedures, templates, tools, and reviews help ensure that all parts of the software are tested using a reliable approach.

Q18. Testing Untested Edge Cases and Boundary Conditions

Answer:

Edge cases are unusual or extreme conditions that may not occur during normal usage. If these cases are ignored, defects may remain hidden until after deployment.

1. Why Edge Cases Are Missed

Edge cases may be missed because of:

- Incomplete requirements
- Limited test coverage
- Focus only on normal inputs
- Lack of boundary testing
- Insufficient test planning
- Time constraints

2. Boundary Value Analysis

Boundary Value Analysis focuses on values at the minimum and maximum limits.

For example, if a system accepts values from **1 to 100**, useful test values are:

- 0
- 1
- 2
- 99
- 100
- 101

This helps identify defects near the boundaries.

3. Practical Example

Suppose an online banking system allows transfers from ₹1,000 to ₹50,000.

The tester should check:

Input	Expected Result
₹999	Rejected
₹1,000	Accepted
₹1,001	Accepted
₹49,999	Accepted
₹50,000	Accepted
₹50,001	Rejected

4. Include Edge Cases

Test design should include:

- Minimum values
- Maximum values

- Zero values
- Negative values
- Empty inputs
- Very large inputs
- Invalid formats
- Network interruptions
- Repeated operations

5. Improvements

The team should:

- Use Boundary Value Analysis.
- Use Equivalence Partitioning.
- Review requirements carefully.
- Maintain edge-case checklists.
- Analyze previous defects.
- Perform regression testing.
- Automate important boundary tests.

Conclusion:

Edge cases are important because failures often occur at the limits of system behaviour. Boundary testing and systematic test design help detect such defects before deployment.

Q19. Defect Documentation and Repository Management

Answer:

Proper defect documentation is essential for tracking software problems from identification to resolution. A defect repository provides a centralized location for storing and analyzing defect information.

1. Problems Caused by Poor Documentation

Poor defect documentation can lead to:

- Lost defects
- Duplicate defect reports
- Difficulty reproducing problems
- Delayed resolution
- Poor communication
- Repeated defects

2. Information to Record

A defect repository should contain:

- Defect ID
- Title
- Description
- Module
- Environment
- Steps to reproduce
- Expected result
- Actual result
- Severity
- Priority
- Assigned person
- Status
- Resolution
- Closure information

3. Structured Repository

A structured repository allows the team to search and analyze defects easily.

For example, defects can be grouped according to:

- Module
- Severity
- Priority
- Defect type
- Release
- Root cause
- Status

4. Practical Example

Suppose a company has many defects related to payment processing.

By analyzing the repository, the team discovers that most payment defects occur after database changes.

The team can then increase database integration testing and regression testing for future releases.

5. Documentation Improvements

The organization should:

- Use a standard defect template.
- Maintain a centralized repository.
- Define severity and priority clearly.
- Attach supporting evidence.
- Track defect status.
- Perform regular defect reviews.
- Analyze historical defect patterns.

6. Benefits

Proper documentation provides:

- Better traceability
- Faster resolution
- Better communication
- Easier defect analysis
- Reduced recurring defects
- Improved test planning

Conclusion:

A well-maintained defect repository improves defect management by providing accurate and organized information. Historical defect analysis can also help the team prevent similar problems in future releases.

Q20. Improving Testing of a Banking Transaction System

Answer:

A banking system must process transactions accurately because failures can result in financial loss and reduced customer confidence. Repeated transaction failures indicate that the existing testing approach is not sufficiently structured.

1. Identify the Problems

Repeated failures may occur because of:

- Incomplete requirements
- Random testing
- Missing boundary tests
- Poor input validation
- Integration problems
- Insufficient regression testing

- Untested network failures
- Inadequate test coverage

2. Redesign the Testing Approach

The team should follow:

Requirement Analysis → Test Planning → Test Case Design → Test Execution → Defect Reporting → Retesting → Regression Testing

3. Test Case Design

Important transaction scenarios include:

Test Case	Condition	Expected Result
TC01	Valid account and sufficient balance	Transaction succeeds
TC02	Insufficient balance	Transaction rejected
TC03	Invalid account	Error message displayed
TC04	Zero transaction amount	Transaction rejected
TC05	Amount exceeds daily limit	Transaction rejected
TC06	Network interruption	Transaction handled safely
TC07	Repeated transaction request	Duplicate transaction prevented

4. Testing Techniques

The team should use:

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Table Testing
- State Transition Testing
- Integration Testing
- Regression Testing
- Performance Testing

5. Practical Example

Suppose a customer transfers ₹5,000, but the application deducts the amount twice because the user presses the transfer button repeatedly.

The tester should reproduce the condition and report the defect.

After correction:

1. Retesting confirms that duplicate transactions are prevented.
2. Regression testing checks other transaction functions.
3. Additional test cases are added to prevent recurrence.

6. Improve Defect Detection

Defect detection can be improved by:

- Increasing test coverage.
- Prioritizing high-risk transactions.
- Automating regression tests.
- Testing network failures.
- Testing boundary values.
- Maintaining a defect repository.
- Performing root-cause analysis.
- Reviewing test cases regularly.

7. Expected Improvements

A structured approach provides:

- Fewer transaction failures
- Better test coverage
- Earlier defect detection
- Reliable transaction processing
- Reduced recurring defects
- Increased customer confidence

Conclusion:

Banking transaction testing should be systematic rather than random. Proper test planning, boundary testing, integration testing, regression testing, and defect analysis can significantly improve transaction reliability and software quality.